Балтийский государственный технический университет
«ВОЕНМЕХ» им. Д. Ф. Устинова

Кафедра О7 «Информационные системы и программная инженерия»

Практическая работа №3
по дисциплине «Информатика: Основы программирования»
на тему «Указатели»

Выполнил:
Студент *Праудин Д.С.*
Группа *0717b*
Преподаватель:
Палехова О.А.

Санкт-Петербург
2021 г.

Задание 1.

Проанализировать текст представленной программы и выдаваемые программой результаты. Объяснить, почему результаты именно такие.

Результаты работы программы:

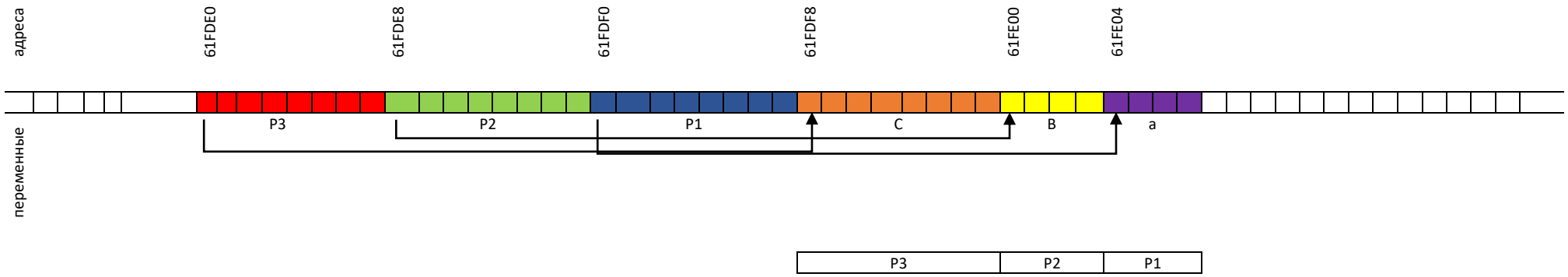
```
a:      int: start address 000000000061FE04 extent 4
b:    float: start address 000000000061FE00 extent 4
c:   double: start address 000000000061FDF8 extent 8

p1: pointer: start address 000000000061FDF0 extent 8
p2: pointer: start address 000000000061FDE8 extent 8
p3: pointer: start address 000000000061FDE0 extent 8

p1: 000000000061FE04 related value 1
p2: 000000000061FE00 related value 2.000000
p3: 000000000061FDF8 related value 3.000000

a=1     b=2.000000      c=3.000000
a=5     b=10.000000     c=1.732051
*p1=5   *p2=10.000000   *p3=1.732051

p1=000000000061FE00     p2=000000000061FE00     p3=000000000061FE00     p4=000000000061FE00
*p1=1092616192  *p2=10.000000   *p3=0.000000    *(float*)p4=10.000000

p1=000000000061FDF0     p2=000000000061FE00     p3=000000000061FDFC
*p1=6422000     *p2=10.000000   *p3=524288.124967

Process returned 0 (0x0)   execution time : 0.092 s
Press any key to continue.
```

Текст программы:

```
#include<stdlib.h>
#include<stdio.h>
#include<math.h>

int main()
{
    /* «Обычные» переменные */
    int a = 1;
    float b = 2;
    double c = 3;
```

```
/* Указатели */
int *p1 = &a;
float *p2 = &b;
double *p3 = &c;
void *p4; /*пустое значение*/
/* Адреса «обычных» переменных и размер выделяемой памяти */
printf("a:      int: start address %p extent %d\n",&a,sizeof(a));
printf("b:    float: start address %p extent %d\n",&b,sizeof(b));
printf("c:   double: start address %p extent %d\n\n",&c,sizeof(c));
/* Адреса указателей и размер выделяемой памяти */
printf("p1: pointer: start address %p extent %d\n",&p1,sizeof(p1));
printf("p2: pointer: start address %p extent %d\n",&p2,sizeof(p2));
printf("p3: pointer: start address %p extent %d\n\n",&p3,sizeof(p3));
/* Значения, на которые ссылаются указатели */
printf("p1: %p related value %d\n",p1,*p1);
printf("p2: %p related value %f\n",p2,*p2);
printf("p3: %p related value %lf\n\n",p3,*p3);
```

Размещение переменных в памяти :



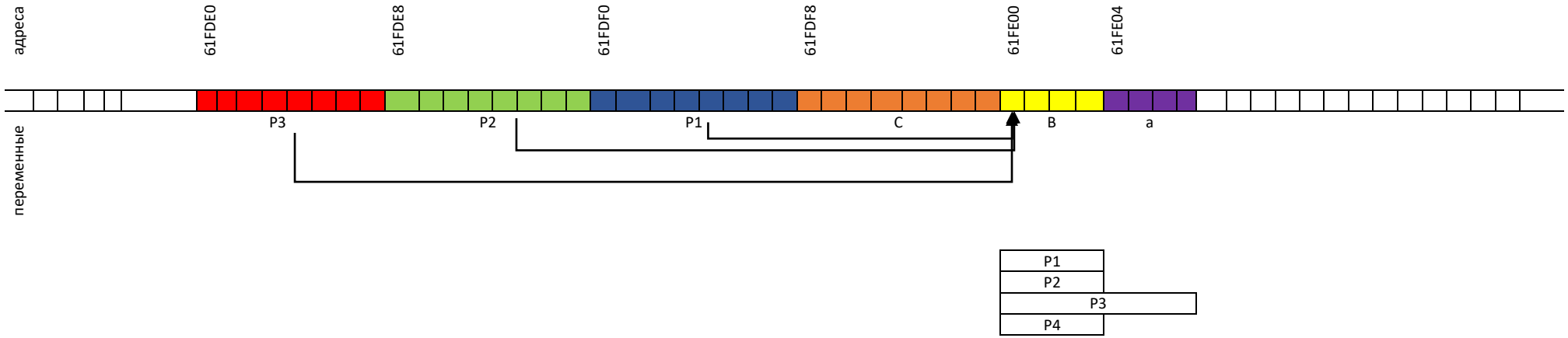
Вывод:

Под переменные типа int и float отведено по 4 байта, под переменную типа double – 8 байт.

Под каждый указатель отведено по 8 байт

Порядок следования байт при записи числа прямой

```
/* Использование указателей в выражениях */
printf("a=%d\tb=%f\tc=%lf\n",a,b,c);
*p1 = 5;
*p2 = *p2 * *p1;
*p3 = sqrt(*p3);
printf("a=%d\tb=%f\tc=%lf\n",a,b,c);
printf("*p1=%d\t*p2=%f\t*p3=%lf\n\n",*p1,*p2,*p3);
/* Присваивание указателей */
p1=(int)p2;
p3=(double)p2;
p4=p2;

printf("p1=%p\tp2=%p\tp3=%p\tp4=%p\n",p1,p2,p3,p4);
printf("*p1=%d\t*p2=%f\t*p3=%f\t*(float*)p4=%f\n\n",*p1,*p2,*p3,*(float*)p4);
```



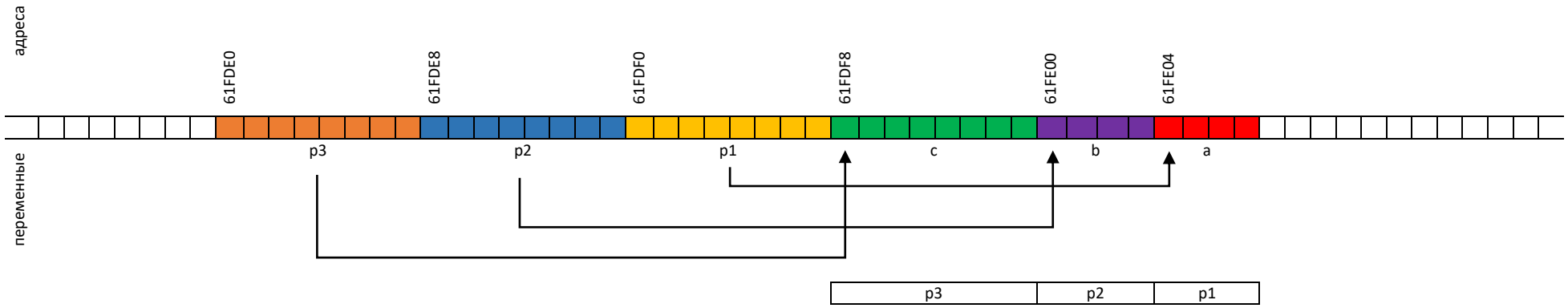
Вывод:

Переменной p1 типа int присвоили значение переменной p2 типа float и, так как целые и вещественные числа в памяти компьютера хранятся по-разному, значение было записано некорректно.

Переменной p3 присвоили значение переменной p2. Теперь в p3 тоже хранится адрес первого байта переменной b, как и в p2, но так как p3 – типа double(8 байт), а p2 – типа float(4 байта), то p3 стала указывать на область памяти, в которой записаны значения двух переменных: b и a. Поэтому при разыменовании невозможно получить корректное значение.

Переменной p4 присвоили значение переменной p2. Указатель p4 типа void, поэтому его сначала привели к типу float, а только потом разыменовали.

```
/* Изменение значений указателей */
p1++;
p3--;
printf("p1=%p\tp2=%p\tp3=%p\n",p1,p2,p3);
printf("*p1=%d\t\t*p2=%f\t*p3=%lf\n",*p1,*p2,*p3);
```



Вывод:

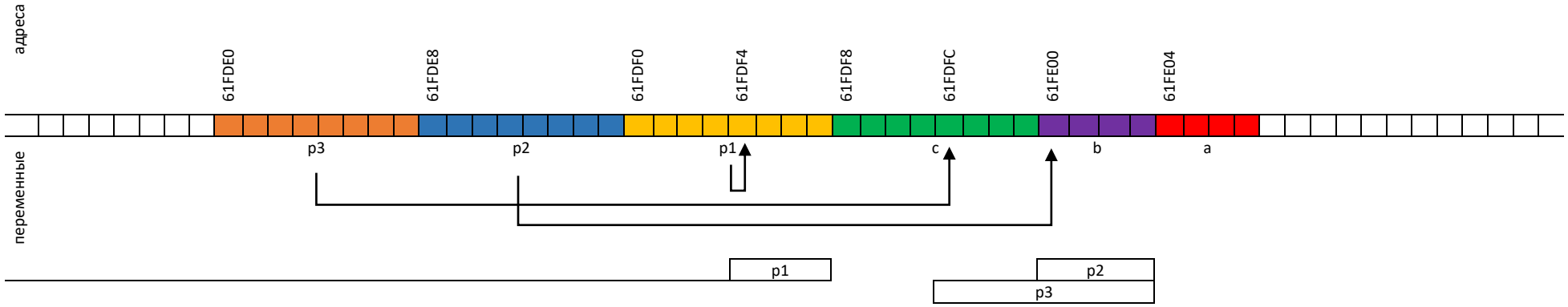
Значение переменной p1 увеличили на 1, но значение адреса, который в ней записан, увеличился на 4. Это произошло потому, что размер одной переменной типа int равен 4 байтам, и изменение адреса, хранящегося в указателе, происходит в единицах умноженных на количество байт, соответствующее переменной данного типа.

С p3 происходит то же самое, её значение уменьшается на 1, а значение адреса, записанного в ней, уменьшается на 8.(Т. к. p3 – типа double, а размер одной переменной типа double равен 8 байт).

Значения, получаемые при разыменовании указателей, стали такими, как были до предыдущего изменения, так как значения указателей тоже стали изначальными.

```
p1 -= 4;
p3 = (double*)&a - 1;
printf("p1=%p\tp2=%p\tp3=%p\n",p1,p2,p3);
```

```
    printf("*p1=%d\t*p2=%f\t*p3=%lf\n",*p1,*p2,*p3);
    return 0;
```




```
}
```

Вывод: Значение указателя p1 = 0 потому что указатель указывает сам на себя, а указатель p3 мы меняем на адрес a - 1

Выводы: от очерёдности инициализации переменных зависит их расположение в памяти. Указатели разных типов указывают на разные по размеру области памяти, поэтому, при смещении, нужно очень внимательно следить за адресами.

Задание 2.

Операционная система *Windows 10*, среда разработки *Code::Blocks*

Проанализировать текст представленной программы, найти в нем синтаксические ошибки и исправить их, в начало программы добавить вывод на экран адресов всех переменных, а в конец – значений всех переменных, проанализировать полученные результаты и объяснить, почему они именно такие. Заменить инструкцию «m+=2;» инструкцией «m++;», проанализировать результат.

Текст измененной программы:

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main()
{
    char *p, c;
    int *a, b;
    float *x, y = 3.5;
    double *m, n;
    printf("%p\n",&p);
    printf("%p\n",&c);
    printf("%p\n",&a);
    printf("%p\n",&b);
    printf("%p\n",&x);
    printf("%p\n",&y);
    printf("%p\n",&m);
    printf("%p\n",&n);
    a = &b;
    printf("  Enter b = ");
    scanf("%d", a);
    printf("a=%p\t*a=%d\tb=%d\n", a, *a, b);
    p =(char *) a;
    c = *p;
    *p = *(p+3);
    *(p+3) = c;
    printf("p=%p\tc=%d\ta=%p\tb=%d\n", p, c, a, b);
    x = &y;
    printf("x=%p\t*x=%f\ty=%f\n", x, *x, y);
    a =(int*)x;
    *a = *x;
    printf("a=%p\t*a=%d\tx=%p\t*x=%f\ty=%f\n", a, *a, x, *x, y);
```

```
a = &b;
y = 12345.6789;
printf("x=%p\t*x=%f\ty=%f\n", x, *x, y);
p =(char*)x;
c=*p;
*p=*(p+3);
*(p+3)=c;
printf("p=%p\tc=%d\tx=%p\ty=%f\n", p, c, x, y);
m = &n;
printf("m=%p\t*m=%lf\tn=%lf\n", m, *m, n);
n = 5.5;
printf("m=%p\t*m=%lf\tn=%lf\n", m, *m, n);
b = n = y = 1.7;
printf("b=%d\ty=%f\tn=%lf\n", b, y, n);
printf("*a=%d\t*x=%f\t*m=%lf\n", *a, *x, *m);
m ++;
printf("n=%lf\tn=%p\tm=%p\n", n, &n, m);
*m = (double)*a - n + (double)*x;
printf("m=%p\t*m=%lf\n", m, *m);
printf("%p\n",p);
printf("%c\n",c);
printf("%p\n",a);
printf("%d\n",b);
printf("%p\n",x);
printf("%f\n",y);
printf("%p\n",m);
printf("%lf\n",n);
return 0;
```

```
}
```

Результаты работы программ:

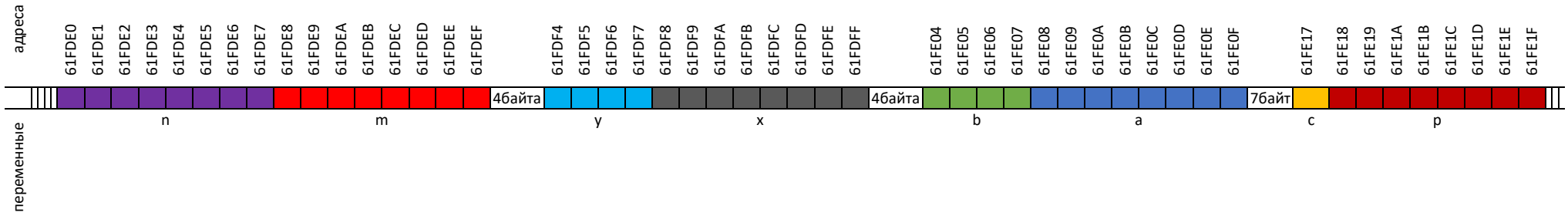
первый вариант (m+=2;)

```
0x7ffdba5fda10
0x7ffdba5fda07
0x7ffdba5fda18
0x7ffdba5fda08
0x7ffdba5fda20
0x7ffdba5fda0c
0x7ffdba5fda28
0x7ffdba5fda30
  Enter b = 12
a=0x7ffdba5fda08        *a=12   b=12
p=0x7ffdba5fda08        c=12    a=0x7ffdba5fda08        b=201326592
x=0x7ffdba5fda0c        *x=3.500000     y=3.500000
a=0x7ffdba5fda0c        *a=3    x=0x7ffdba5fda0c        *x=0.000000     y=0.000000
x=0x7ffdba5fda0c        *x=12345.678711 y=12345.678711
p=0x7ffdba5fda0c        c=-73   x=0x7ffdba5fda0c        y=-0.000011
m=0x7ffdba5fda30        *m=0.000000     n=0.000000
m=0x7ffdba5fda30        *m=5.500000     n=5.500000
b=1     y=1.700000      n=1.700000
*a=1    *x=1.700000     *m=1.700000
n=1.700000      n=0x7ffdba5fda30        m=0x7ffdba5fda40
m=0x7ffdba5fda40        *m=1.000000
0x7ffdba5fda0c
0x7ffdba5fda08
1
0x7ffdba5fda0c
1.700000
0x7ffdba5fda40
1.700000


...Program finished with exit code 0
Press ENTER to exit console.
```

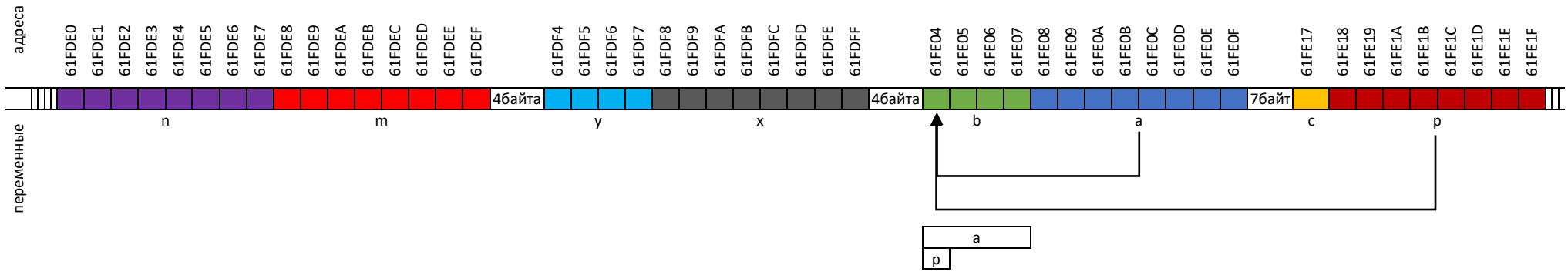
второй вариант (m++;)

```
0x7ffcb8c77f20
0x7ffcb8c77f17
0x7ffcb8c77f28
0x7ffcb8c77f18
0x7ffcb8c77f30
0x7ffcb8c77f1c
0x7ffcb8c77f38
0x7ffcb8c77f40
  Enter b = 12
a=0x7ffcb8c77f18        *a=12   b=12
p=0x7ffcb8c77f18        c=12    a=0x7ffcb8c77f18        b=201326592
x=0x7ffcb8c77f1c        *x=3.500000     y=3.500000
a=0x7ffcb8c77f1c        *a=3    x=0x7ffcb8c77f1c        *x=0.000000     y=0.000000
x=0x7ffcb8c77f1c        *x=12345.678711 y=12345.678711
p=0x7ffcb8c77f1c        c=-73   x=0x7ffcb8c77f1c        y=-0.000011
m=0x7ffcb8c77f40        *m=0.000000     n=0.000000
m=0x7ffcb8c77f40        *m=5.500000     n=5.500000
b=1     y=1.700000      n=1.700000
*a=1    *x=1.700000     *m=1.700000
n=1.700000      n=0x7ffcb8c77f40        m=0x7ffcb8c77f48
m=0x7ffcb8c77f48        *m=1.000000
0x7ffcb8c77f1c
0x7ffcb8c77f18
1
0x7ffcb8c77f1c
1.700000
0x7ffcb8c77f48
1.700000
*** stack smashing detected ***: terminated


...Program finished with exit code 0
Press ENTER to exit console.
```

Размещение переменных в памяти:

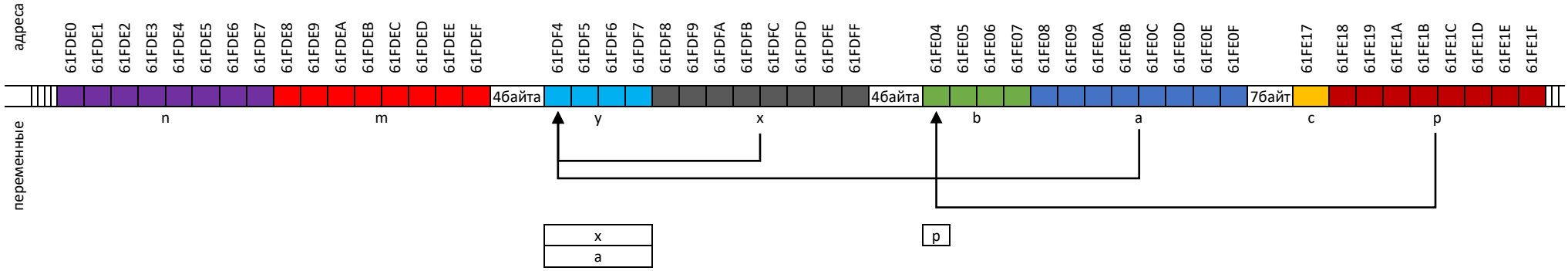

адреса
61FDE0
61FDE1
61FDE2
61FDE3
61FDE4
61FDE5
61FDE6
61FDE7
61FDE8
61FDE9
61FDEA
61FDEB
61FDEC
61FDED
61FDEE
61FDEF
4байта
61FDF4
61FDF5
61FDF6
61FDF7
61FDF8
61FDF9
61FDFA
61FDFB
61FDFC
61FDFD
61FDFE
61FDFF
4байта
61FE04
61FE05
61FE06
61FE07
61FE08
61FE09
61FE0A
61FE0B
61FE0C
61FE0D
61FE0E
61FE0F
7байт
61FE17
61FE18
61FE19
61FE1A
61FE1B
61FE1C
61FE1D
61FE1E
61FE1F
n
m
y
x
b
a
c
p
переменные

```
p = (char*)a;
c = *a;
*p = *(p + 3);
*(p+3) = c;
a = &b;
printf("p=%p\tc=%d\ta=%p\tb=%d\n", p, c, a, b);
```



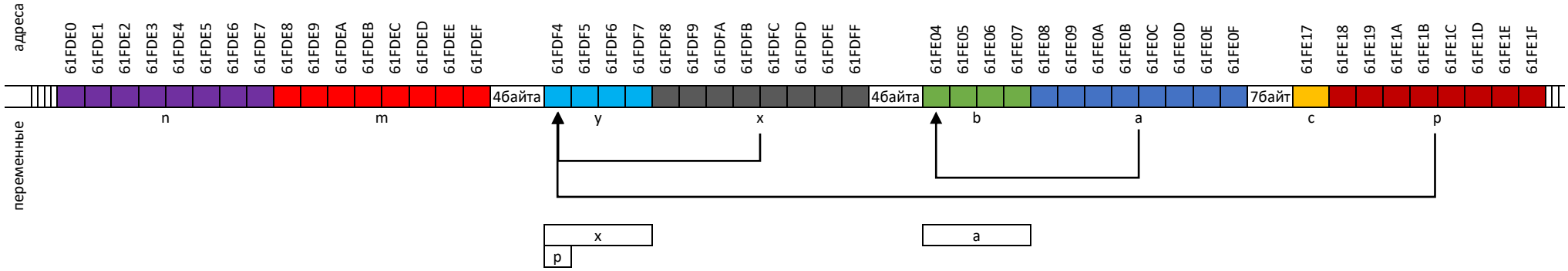
Объяснение: Перестановка значений первого и последнего байта переменной b, переменная c использовалась, как промежуточное хранилище перемещаемого значения, и после всех действий она осталась равна значению первого байта переменной b

```
x = &y;
printf("x=%p\t*x=%f\ty=%f\n", x, *x, y);
a = (int*)x;
*a = *x;
printf("a=%p\t*a=%d\tx=%p\t*x=%f\ty=%f\n",a, *a, x, *x, y);
```



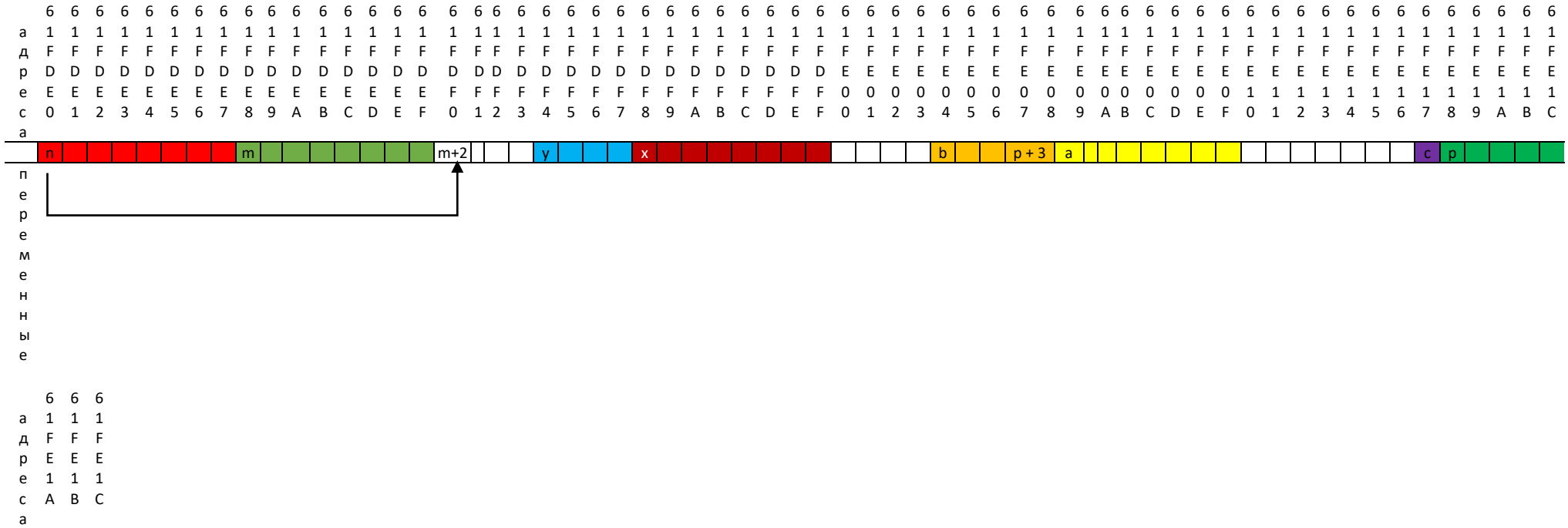
Объяснение: a = не тому числу которое ему присваиваем потому что это целочисленный тип и, поскольку мы пытаемся присвоить ему вещественный тип, машина не видит деления на значения на мантиссу и целую часть поэтому воспринимает число в двоичной системе счисления как целое и следовательно выводит его.

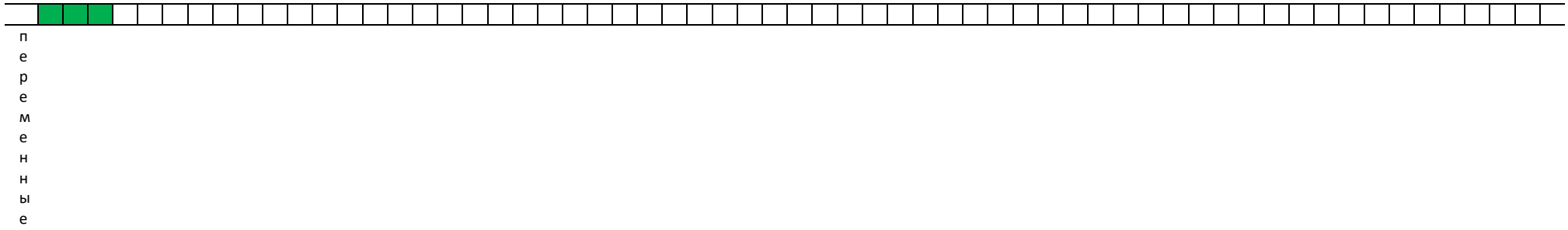
```
p = (char*)x;
c = *p;
*p = *(p+3);
*(p + 3) = c;
printf("p=%p\tc=%d\tx=%p\ty=%f\n", p, c, x, y);
```



Объяснение: Перестановка значений первого и последнего байта переменной y, переменная c использовалась, как промежуточное хранилище перемещаемого значения, и после всех действий она осталась равна значению первого байта переменной y*

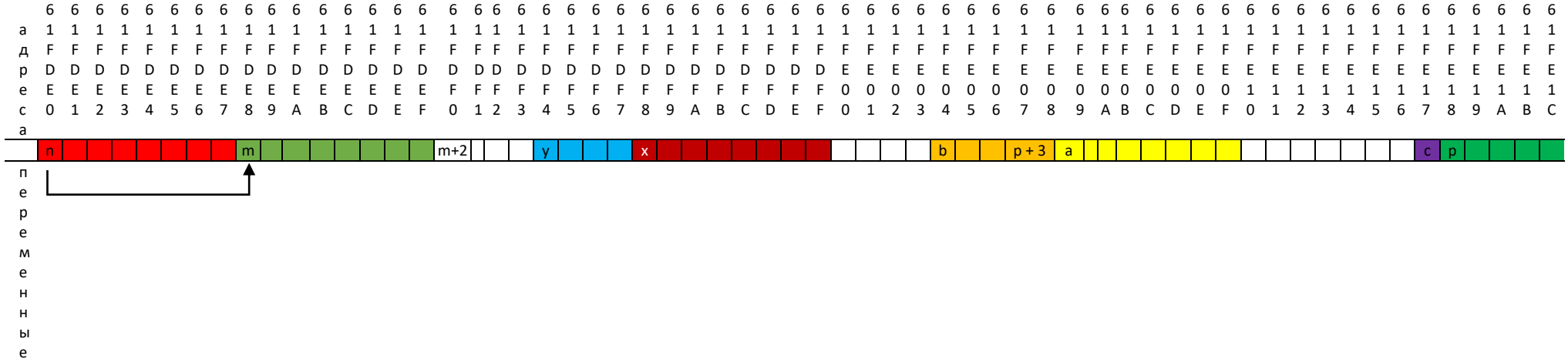
```
m = m + 2;
printf("n=%lf\t n=%p\t m=%p\n", n, &n, m);
*m = (double)*a - n + (double)*x;
printf("*m=%lf\t\n", *m);
```


адреса
6 6 6



Объяснение: Сначала выполняем инструкцию m+2 после чего задаём указателю *m значение выражения “(double)*a - n + (double)*x ”.

```
m ++;
printf("n=%lf\t n=%p\t m=%p\n", n, &n, m);
*m = (double)*a - n + (double)*x;
printf("*m=%lf\t\n", *m);
```



После замены инструкции «m+=2;» инструкцией «m++;» ++;»: Переменная m начинает указывать сама на себя в следствии чего происходит ошибка.

3.Объявить по две переменные типов *char, int* и *double*, а также указатель на *char*. Вывести на экран размеры и адреса всех переменных, начертить схему расположения переменных в памяти. Поменять порядок объявления переменных (например, *int, char, double, char*, char, double, int*). Запустить программу повторно, проанализировать, что изменилось. Задать переменной типа *int* такое значение, чтобы значение каждого байта было уникальным, использовать для этого шестнадцатеричную константу. Записать адрес этой переменной в указатель на *char* и с его помощью вывести на экран содержимое каждого байта (тоже в шестнадцатеричной системе счисления). Проанализировать, прямой или обратный порядок расположения байт при записи числа применяется в используемой системе.

Повторить выполнение этого задания на компьютере с другой операционной системой и/или другой IDE (можно использовать домашний компьютер или онлайн-компилятор). Сравнить результаты работы программы на разных платформах, сделать выводы.

1. Операционная система *widows 10* среда разработки *Code::Blocks*

Текст программы (первый порядок объявления переменных):

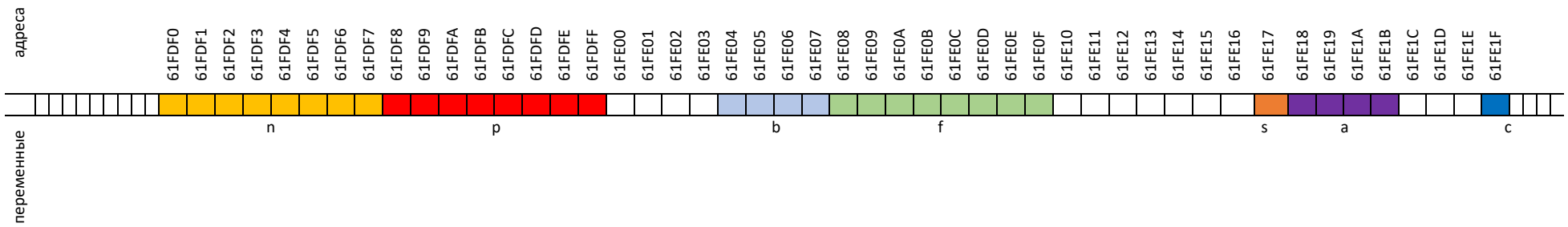
```
#include <stdio.h>
#include <stdlib.h>
int main()
{
char c;
int a;
char s;
double f;
int b;
char *p;
double n;
printf("c: char: start address %p extent %ld\n",&c,sizeof(c));
printf("s: char: start address %p extent %ld\n",&s,sizeof(s));
printf("a: int: start address %p extent %ld\n",&a,sizeof(a));
```

```
printf("b: int: start address %p extent %ld\n",&b,sizeof(b));
printf("f: double: start address %p extent %ld\n",&f,sizeof(f));
printf("n: double: start address %p extent %ld\n",&n,sizeof(n));
printf("p: pointer: start address %p extent %ld\n\n",&p,sizeof(p));
a = 0x50304020;
printf("a = %x\n\n", a);
p = (char*)&a;
for(b=0;b<4;b++)
{
printf("%x\t", *p);
p++;
}
return 0;
}
```

Результаты работы программы:

```
c: char: start address 000000000061FE1F extent 1
s: char: start address 000000000061FE17 extent 1
a: int: start address 000000000061FE18 extent 4
b: int: start address 000000000061FE04 extent 4
f: double: start address 000000000061FE08 extent 8
n: double: start address 000000000061FDF0 extent 8
p: pointer: start address 000000000061FDF8 extent 8

a = 50304020

20      40      30      50
Process returned 0 (0x0)   execution time : 0.067 s
Press any key to continue.
```

Размещение переменных в памяти:



Текст программы (второй порядок объявления переменных):

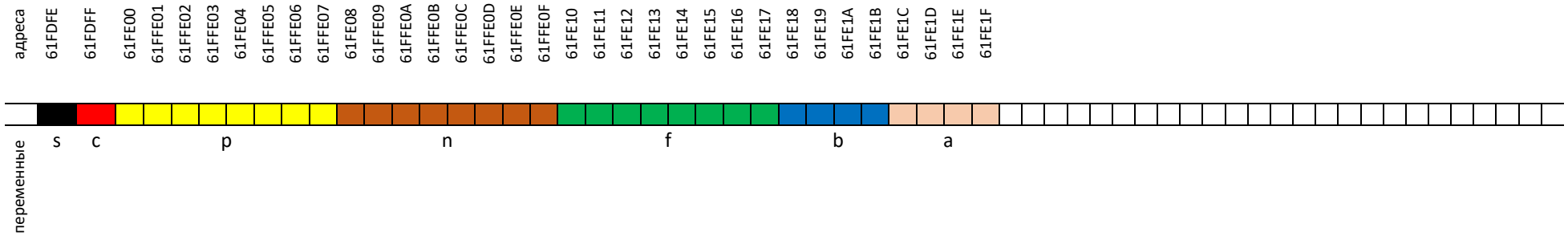
```
#include <stdio.h>
#include <stdlib.h>
int main()
{
int a, b;
double f, n;
char *p;
char c, s;
printf("a: int: start address %p extent %ld\n",&a,sizeof(a));
printf("b: int: start address %p extent %ld\n",&b,sizeof(b));
printf("f: double: start address %p extent %ld\n",&f,sizeof(f));
printf("n: double: start address %p extent %ld\n",&n,sizeof(n));
```

```
printf("p: pointer: start address %p extent %ld\n",&p,sizeof(p));
printf("c: char: start address %p extent %ld\n",&c,sizeof(c));
printf("s: pointer: start address %p extent %ld\n\n",&s,sizeof(s));
a = 0x50304020;
printf("a = %x\n\n", a);
p = (char*)&a;
for(b=0;b<4;b++)
{
printf("%x\t", *p);
p++;
}
return 0;
}
```

Результаты работы программы:

```
a: int: start address 000000000061FE1C extent 4
b: int: start address 000000000061FE18 extent 4
f: double: start address 000000000061FE10 extent 8
n: double: start address 000000000061FE08 extent 8
p: pointer: start address 000000000061FE00 extent 8
c: char: start address 000000000061FDFF extent 1
s: pointer: start address 000000000061FDFE extent 1

a = 50304020

20      40      30      50
Process returned 0 (0x0)   execution time : 0.016 s
Press any key to continue.
```

Размещение переменных в памяти *:*



Выводы:

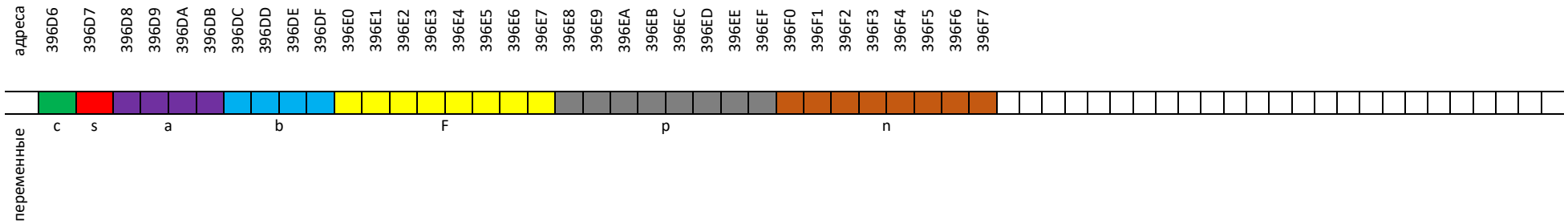
От порядка объявления переменных зависит их расположения в памяти. При объявлении переменные занимают ближайшую пустую ячейку в памяти из за чего и разнятся выше показанные примеры.
Порядок следования байт при записи числа прямой

1.Операционная система **windows 10**, среда разработки ***онлайн-компилятор online gdb***

Результаты работы программы при первом порядке объявления переменных:

```
c: char: start address 0x7ffdc9c396d6 extent 1
s: char: start address 0x7ffdc9c396d7 extent 1
a: int: start address 0x7ffdc9c396d8 extent 4
b: int: start address 0x7ffdc9c396dc extent 4
f: double: start address 0x7ffdc9c396e0 extent 8
n: double: start address 0x7ffdc9c396f0 extent 8
p: pointer: start address 0x7ffdc9c396e8 extent 8

a = 50304020

20      40      30      50

...Program finished with exit code 0
Press ENTER to exit console.
```

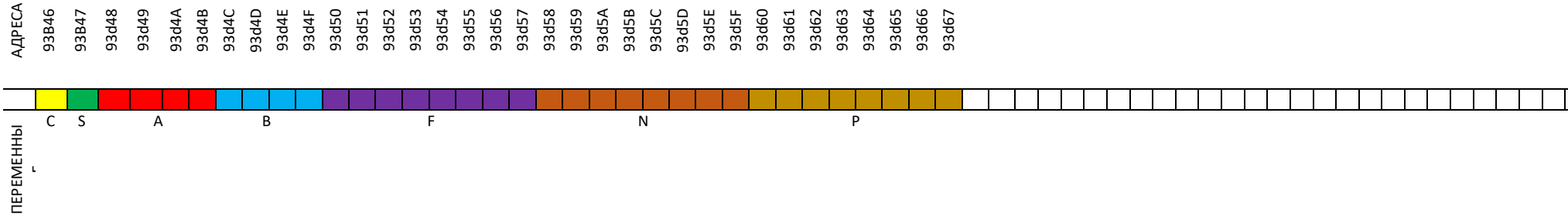
Размещение переменных в памяти *:*



Результаты работы программы при втором порядке объявления переменных:

```
a: int: start address 0x7ffc8f893d48 extent 4
b: int: start address 0x7ffc8f893d4c extent 4
f: double: start address 0x7ffc8f893d50 extent 8
n: double: start address 0x7ffc8f893d58 extent 8
p: pointer: start address 0x7ffc8f893d60 extent 8
c: char: start address 0x7ffc8f893d46 extent 1
s: pointer: start address 0x7ffc8f893d47 extent 1

a = 50304020

20      40      30      50

...Program finished with exit code 0
Press ENTER to exit console.
```

Размещение переменных в памяти :



Выводы: переменные расставляются в памяти в порядке возрастания размера выделяемой памяти в зависимости от типа(сначала char(1 байт), потом int(4 байта) и т.д.). Вторая по приоритету расстановка - по очереди объявления(Переменная, которая объявлена позже будет иметь больший адрес.

Порядок следования байт при записи числа *обратный*

Выводы: Online IDE GDB: при объявлении переменные сортируются по размеру, зависящему от типа, после чего по очереди объявления. Пустых пространств между переменными разных типов нет.